PIM Design Document

Software requirements specification

Functional Requirements:

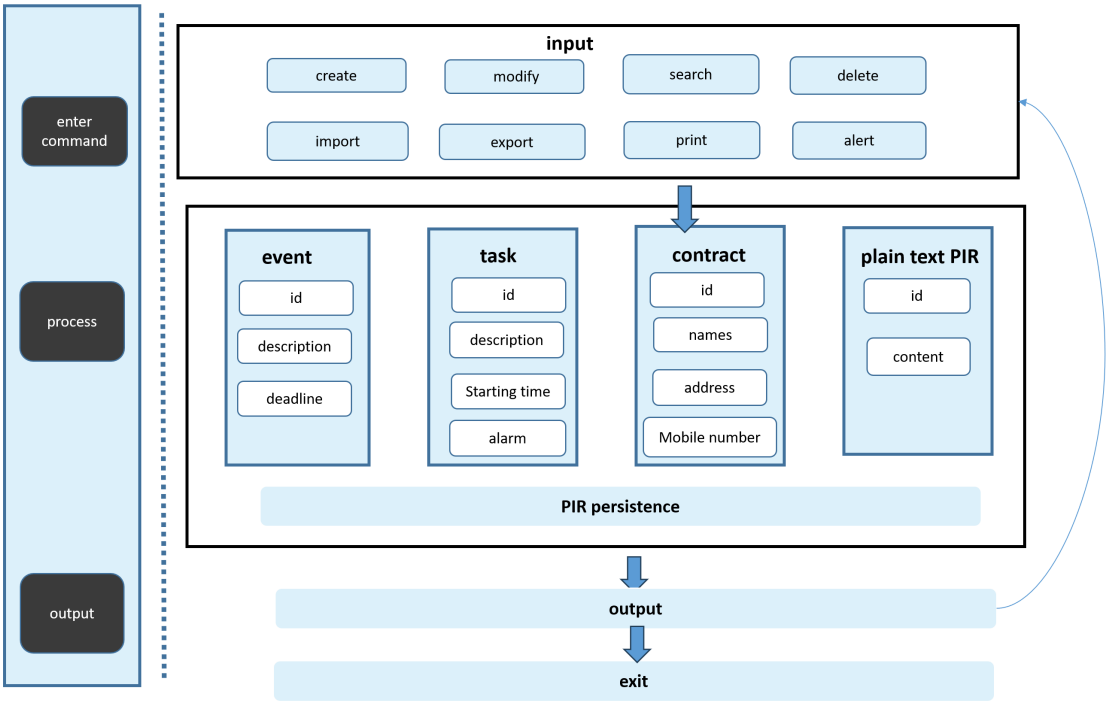
1. PIR creation:
   - The PIM shall allow the user to create new PIRs of different types (event, task, contact).
   - The PIM shall enable the user to create new plain-text PIRs.
   - The PIM shall allow the user to create new task PIRs with corresponding descriptions and deadlines.
   - The PIM shall allow users to create new event PIRs with corresponding descriptions, starting times, and alarms.
   - The PIM shall allow the user to create new contact PIRs with corresponding names, addresses, and mobile numbers.
2. PIR management:
   - The PIM shall allow the user to modify the data in existing PIRs.
   - The PIM shall allow the user to search for PIRs based on criteria concerning their types and the data stored in their fields. Search Criteria (a piece of text (stored in a note, a description, a name, an address, or a mobile number) contains a string, whether a time (stored in a deadline, a starting time, or an alarm) is before (<), after (>), or equal to (=) another given point in time, or whether a condition combining multiple other conditions via logical connectors and (&&), or (||) and negation (!) is satisfied.)
3. The PIM shall allow the user to print out detailed information about a specific PIR or all PIRs.
4. The PIM should alert the user if a task type PIR meets its deadline, or an event type PIR meets its alarm date.
5. The PIM shall allow the user to delete a specified PIR.

6. PIR import and export:
    - The PIM shall allow the user to store the PIRs in a file with the extension name .pim.
    - The PIM shall allow the user to load the PIRs from a file with the extension name .pim.

Non-Functional Requirements::

1. The system should have efficient search functionality to quickly retrieve relevant PIRs, with a response time of less than 1 second.
2. The system should ensure the security and privacy of PIRs stored within the application, with no known security vulnerabilities and no unauthorized access to PIRs.
3. The system should have reliable data storage to prevent loss of PIRs, with a data loss rate of less than 0.01%.
4. The system should have backup and restore capabilities for PIR data, with a restore time of less than 2 hours.
5. The system should be compatible with standard operating systems, including Windows, macOS, and Linux.
6. The system should have responsive performance, providing quick response times for user interactions, with a page load time of less than 3 seconds.
7. The system should be scalable to handle many PIRs, up to 1 million PIRs, without any performance degradation.
8. The system should have a robust error-handling mechanism to handle unexpected situations without system crashes or data loss.
9. The system should provide adequate documentation and user guides for easy understanding and usage, with a satisfaction rating of at least 90% among users.
10. The system should be easily maintainable, allowing for future updates and enhancements, with a mean time for repairs of less than 1 hour.

Architecture



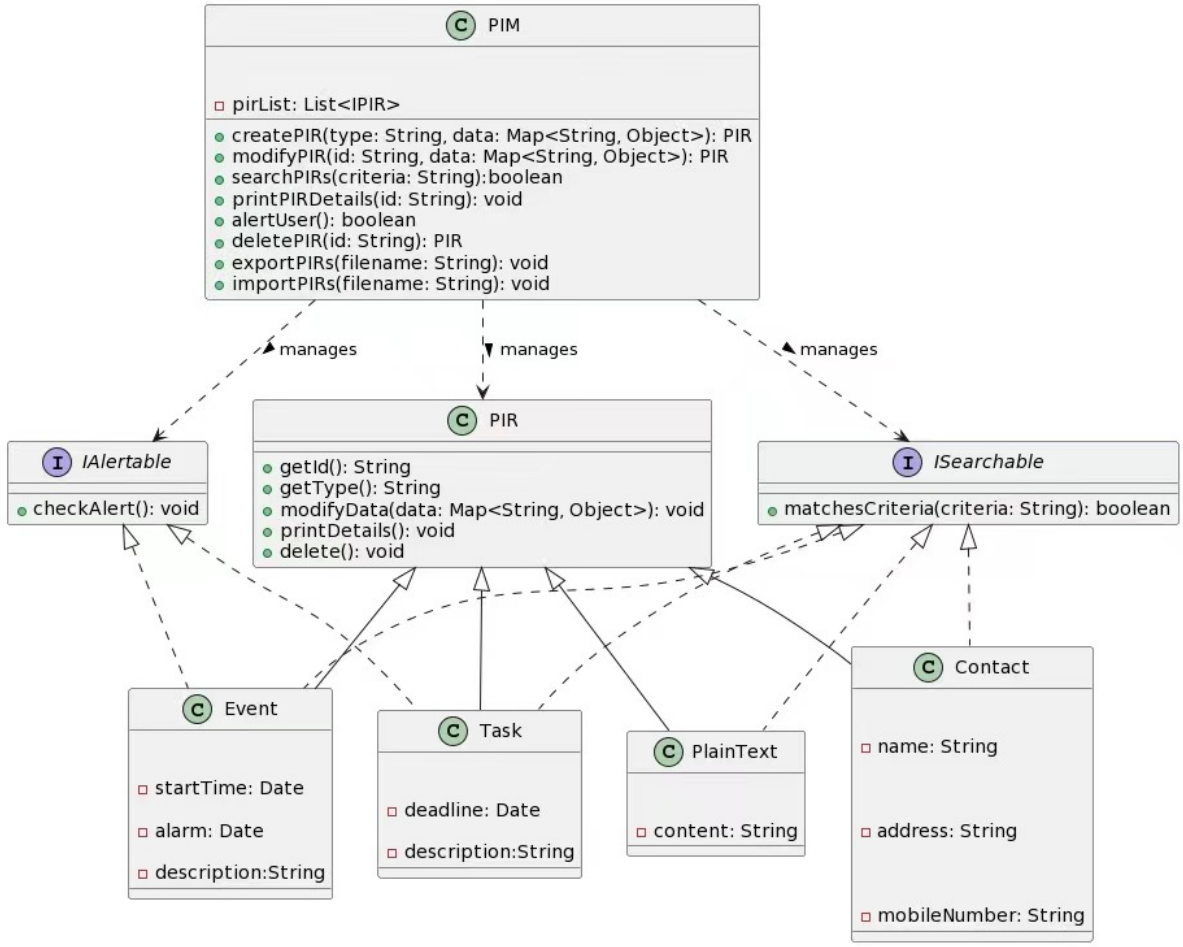
The architecture of the PIM system consists of three layers: instruction input, PIM instruction processing, and result return.

Instruction input module: Users can input the desired PIM operation in the PIM terminal according to the instruction rules, which includes instructions such as create modify search print delete import export alert help, etc

PIM instruction processing: After receiving user input instructions, PIM will parse and convert them into PIM related operations. For example, if you enter create, the corresponding PIR class will be called based on the type value to create, and the newly created PIR will be added to the PIR list. The underlying targets of PIM operations are four types of PIRs.

Output module: PIM parses the meaning of the received user instructions and returns the corresponding operation results to only the PIM running terminal. Users can choose to continue executing other instructions or exit the program.

Structure of and relationship among major code components



In this project, we defined four types of PIR subclasses, namely Contact Event PlainText Task, which inherit from the base class PIR. In addition, we have defined two interface definitions: IAlertable and ISearchable. The IAlertable interface is used to check whether the PIR meets the alarm requirements, and the ISearchable interface is used to check whether the PIR meets the search criteria given by the user.

In PIM.class, it is used to interact with users. PIM receives user input instructions and parses them to perform corresponding operations.

PIR.class

PIR.class is the base class for Persional information record. The PIR.class defines various types of common attributes and member methods in records.

The member field are as follows:

field	type	remark
id	String	Each record has a unique identification code
type	String	The type of each record, with values of contact, event, plainText or task

The member functions are as follows:

functions	Return Type	remark
getId()	String	Obtain the ID value of each record.
getType()	String	Obtain the type value of each record.
modifyPIR(Map<String, Object> data)	PIR	Modify the PIR related field function, which is a virtual function. After subclasses inherit this class, they need to implement their respective function body.
printDetails()	void	Print PIR Details. This function is a virtual function, and subclasses that inherit this class need to implement their respective function printing.

All subclasses that inherit the PIR class need to implement relevant virtual functions (modifyPIRand printDetails).

IAlterable interface

The methods in the IAlterable interface are as follows:

functions	Return Type	remark
checkAlert()	boolean	Check if the PIR needs to remind the user

The methods in the interface cannot be implemented in the interface, and can only be implemented by the class that implements the interface. In our project, all types of PIRs have implemented this interface.

ISearchable interface

The methods in the ISearchable interface are as follows:

functions	Return Type	说明

functions	Return Type	说明
matchesCriteria(String criteria)	boolean	Check if the current PIR record meets the search criteria.

The methods in the interface cannot be implemented in the interface, and can only be implemented by the class that implements the interface. In our project, event and task type PIRs implement this interface because there are time related fields in these two types of PIRs.

Contact.class

Contact.class represents the PIR related information of the contact type, which inherits from PIR.class and has all the attributes and functions related to the PIR base class. PIM.class implements interface ISearchable interface.

In addition to the relevant attributes of the base class, this class has unique fields.

field	type	remark
name	String	Contact name
address	String	Contact address
mobileNumber	String	Contact mobile number

functions	Return Type	remark
modifyPIR(Map<String, Object> data)	PIR	Functions for modifying contact related fields
printDetails()	void	Functions for print contact detail info
matchesCriteria(String criteria)	boolean	Check if the contact PIR record meets the search criteria.

Event.class

Event.class represents the PIR related information of the event type, which inherits from PIR.class and has all the attributes and functions related to the PIR base class. PIM.class implements interface ISearchable and IAlterable .

In addition to the relevant attributes of the base class, this class has unique fields.

field	type	remark
description	String	Event description
startTime	Date	Event start time
alarm	Date	Event alert time

functions	Return Type	remark
modifyPIR(Map<String, Object> data)	PIR	Functions for modifying event related fields.
printDetails()	void	Functions for print event detail info.
matchesCriteria(String criteria)	boolean	Check if the event PIR record meets the search criteria.
checkAlert()	boolean	Check if the event PIR needs to alert the user. The PIM should alert the user if an event type PIR meets its alarm date.

PlainText.class

PlainText.class represents the PIR related information of the plainText type, which inherits from PIR.class and has all the attributes and functions related to the PIR base class. PIM.class implements interface ISearchable.

In addition to the relevant attributes of the base class, this class has unique fields.

field	type	remark
content	String	PlainText content

functions	Return Type	remark
modifyPIR(Map<String, Object> data)	PIR	Functions for modifying plain Text related fields.
printDetails()	void	Functions for print plainText detail info.
matchesCriteria(String criteria)	boolean	Check if the plainText PIR record meets the search criteria.

Task.class

Event.class represents the PIR related information of the event type, which inherits from PIR.class and has all the attributes and functions related to the PIR base class. PIM.class implements interface ISearchable and IAlterable .

In addition to the relevant attributes of the base class, this class has unique fields.

field	type	remark
description	String	Task description
deadline	Date	Task deadline time.

functions	Return Type	remark
modifyPIR(Map<String, Object> data)	PIR	Functions for modifying task related fields.
printDetails()	void	Functions for print task detail info.
matchesCriteria(String criteria)	boolean	Check if the task PIR record meets the search criteria.
checkAlert()	boolean	Check if the task PIR needs to alert the user. The PIM should alert the user if a task type PIR meets its deadline.

PIM.class

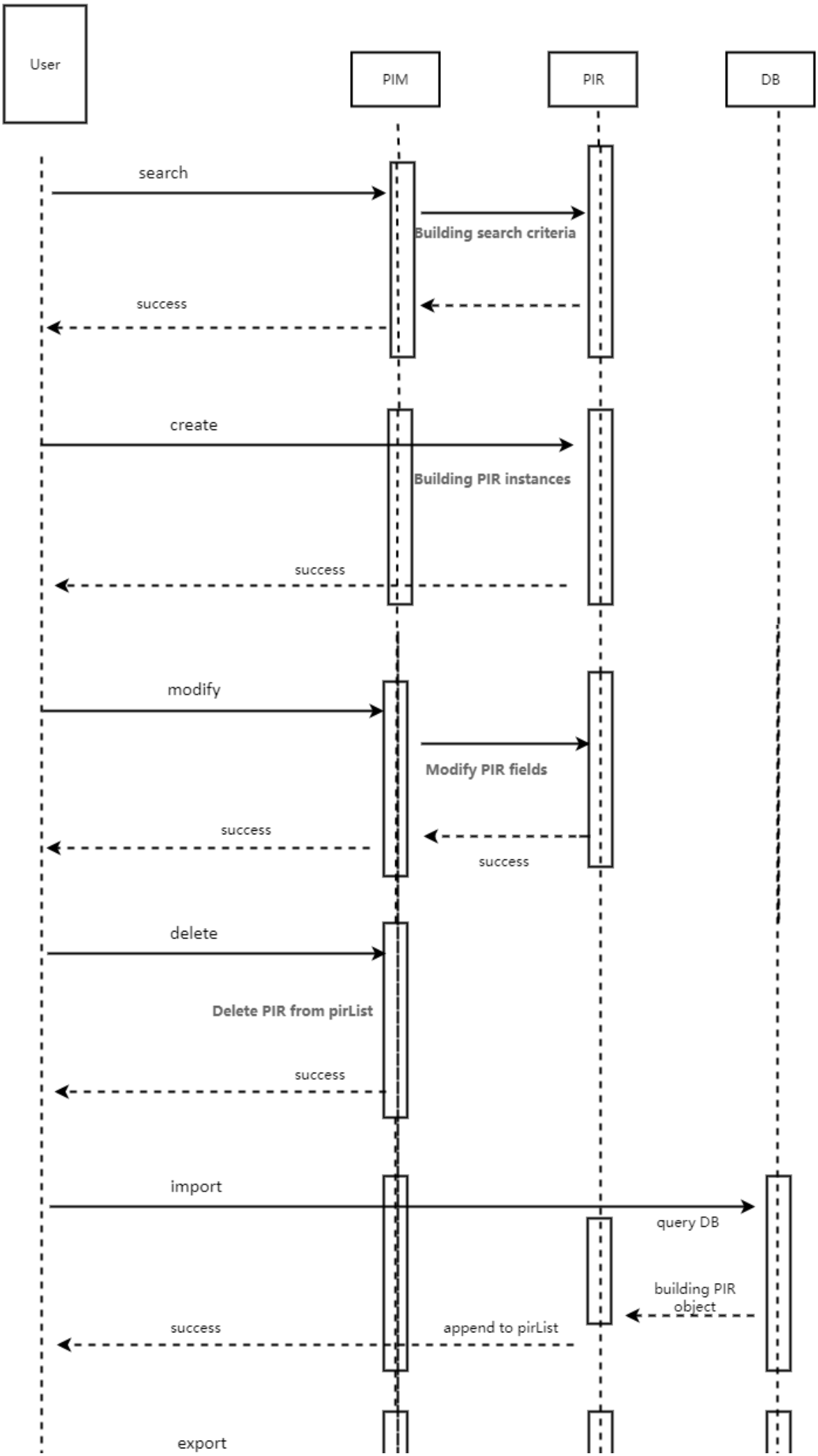
In PIM.class, it is used to interact with users. PIM receives user input instructions and parses them to perform corresponding operations.

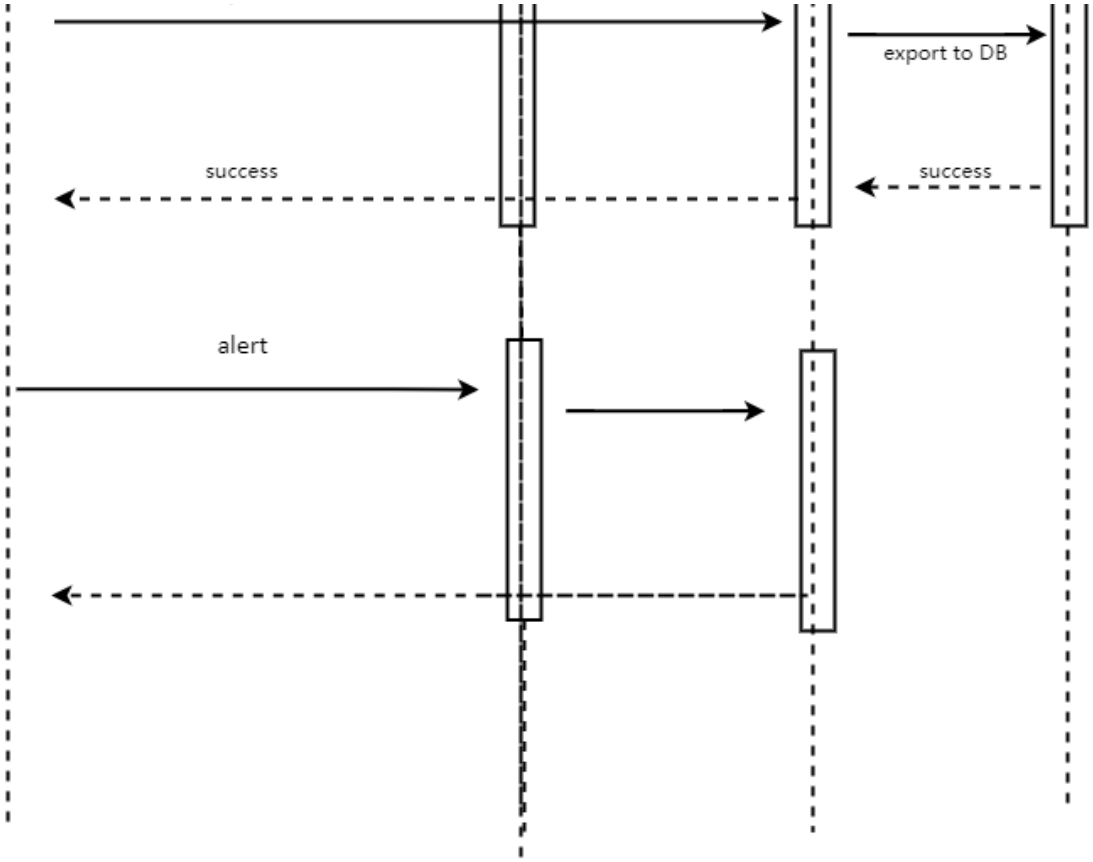
field	type	remark
pirList	LinkedList	PIR record storage list.

functions	Return Type	remark
createPIR(String type, Map<String, Object> data)	PIR	Create a PIR record with parameters of PIR type and a Map composed of PIR attribute names and values. The return value is the PIR object created
modifyPIR(String type, Map<String, Object> data)	PIR	Modify a PIR record with parameters such as the PIR id value that needs to be modified, and a Map that combines the attribute names and values that need to be modified for PIR.The return value is the modified PIR object.
searchPIRs(String criteria)	boolean	Search for PIRs that meet the query criteria and print relevant search results. If the search results are found, return True; otherwise, return False.
printPIRDetails(String id)	void	Print detailed information of PIR The parameter is the PIR id value that needs to be printed. If the id value is "all", all PIR records will be printed.
deletePIR(String id)	PIR	Delete the PIR record, with the parameter being the PIR id value that needs to be deleted. The return value is the deleted PIR object.
alertUser()	boolean	The PIM should alert the user if a task type PIR meets its deadline, or an event type PIR meets its alarm date. If a PIR record that requires an alert is found, return True Otherwise, return False.

functions	Return Type	remark
exportPIRs(String filename)	void	The PIM shall allow the user to store the PIRs in a file. The parameter is the export file name.
importPIRs(String filename)	void	The PIM shall allow the user to load the PIRs from a file. The parameter is the import file name.
usage()	void	Print usage infomation.

Example use

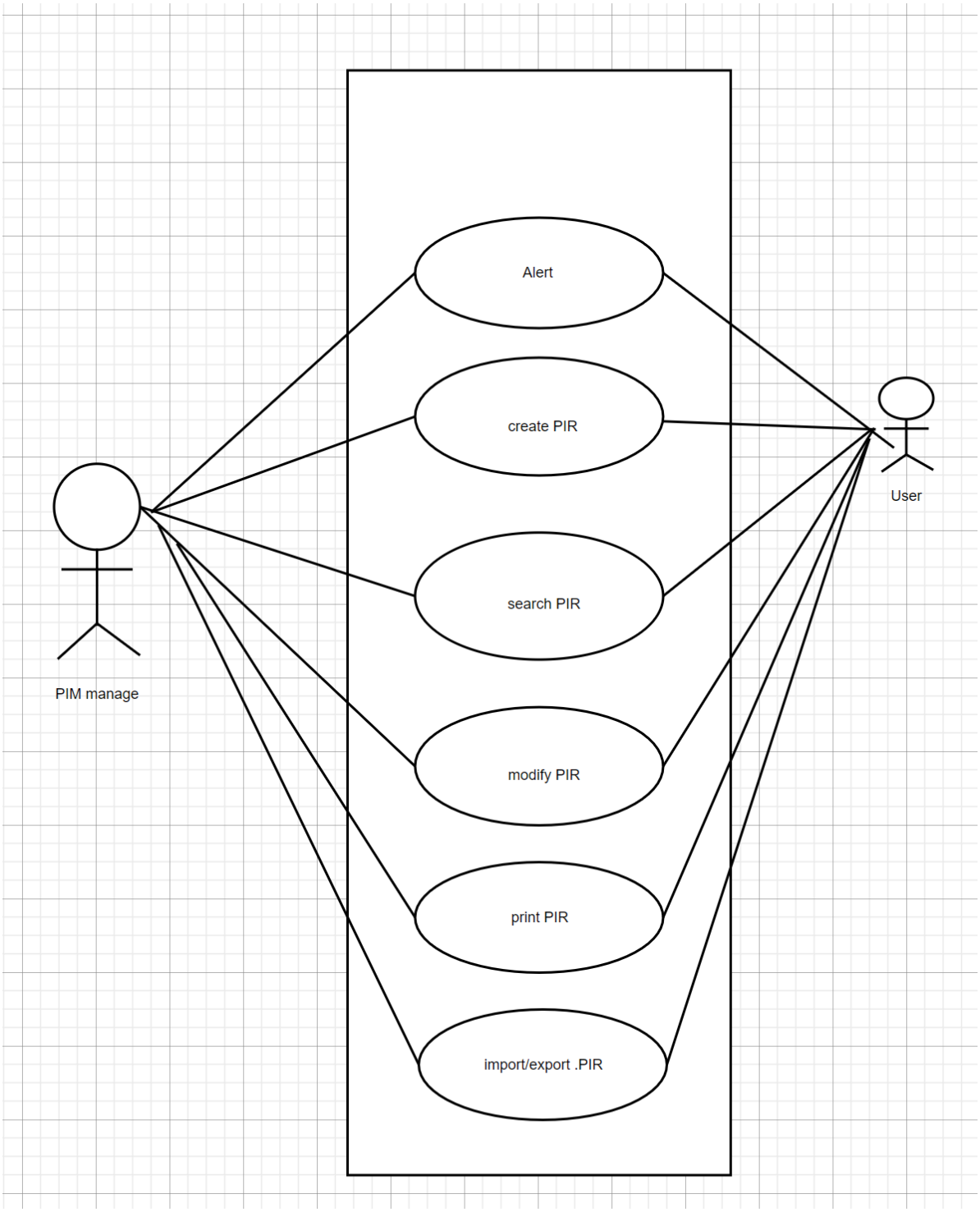

User
PIM
PIR
DB
search
Building search criteria
success
create
Building PIR instances
success
modify
Modify PIR fields
success
success
delete
Delete PIR from pirList
success
import
query DB
building PIR object
success
append to pirList
export



The subjects involved in the timing diagram have four modules, namely **user**, **PIM**, **PIR**, and **database**. Among them, users input instructions according to instruction rules, and the PIM module will receive these instructions and parse them. Some instructions (such as create, modify, search, print, alert etc) require operations on each PIR record.

There are some differences between import and export instructions, both of which involve interacting with the database (where the database is the file that saves the PIR). After the user enters the import instruction, PIM will parse the instruction and read the data from the database name passed in by the user for deserialization (i.e. creating a PIR object that exists in the new database), and then store the created PIR object in the PIR list.

User case



Use cases analyze and examine the behavior of the system to be developed from the perspective of external users and peripheral systems, and describe the functional characteristics provided by the system externally through the interaction relationship between participants (either end users or peripheral systems) - this interaction relationship between participants and system functional characteristics is called use cases.

The participants involved in the PIM system are users and the PIM system itself. The use case for interacting with the PIM system includes "create", "modify", "print","search" "alert", "import" or "export". The user transmits the instruction to the PIM system through the command line, which parses and processes the instruction and returns the processing result to the user. The user and the system repeatedly repeat the above process to create a user exit from the system.